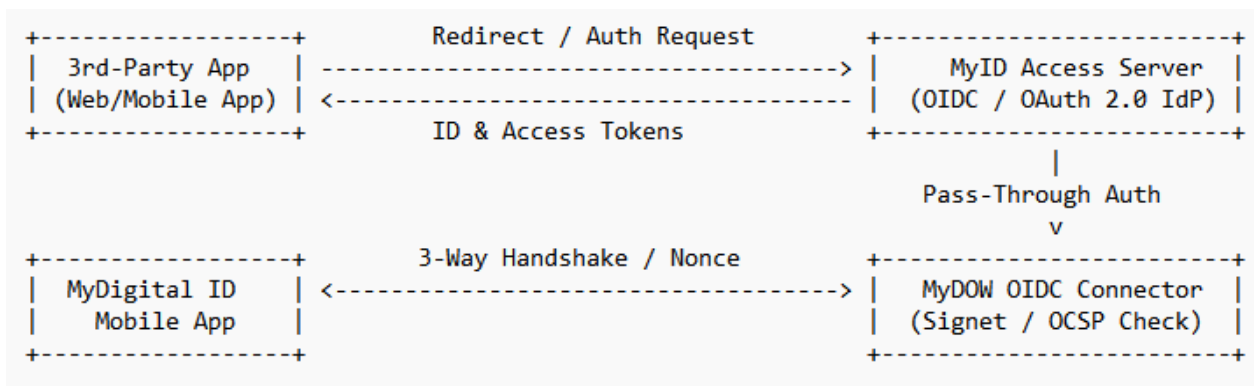


MyID Access Integration Guide

This guide provides the full operational breakdown and integration steps for implementing **MyID Access** (MyDigital ID Single Sign-On (SSO)). It translates the complex identity flows into actionable steps for engineers while maintaining non-technical clarity.

1. Architectural & Protocol Overview

MyDigital ID SSO relies on **Apache Keycloak** as the central Identity and Access Management (IAM) hub.



Protocol Flow Explained

1. **User Request:** The user initiates a login attempt on the 3rd-party application.
2. **Redirection:** The 3rd-party app redirects the browser/app to MyID Access Server using the **OpenID Connect (OIDC)** Authorization Code Flow (scope=openid).
3. **Identity Broker:** Keycloak hands off authentication to the **MyDOW Connector** (custom Identity Provider plugin).
4. **QR / Deep-Link Handshake:** MyDOW displays a QR code (desktop) or triggers a deep-link URI (mobile) tied to a unique WebSocket nonce channel.
5. **MyDigital ID Mobile Verification:**
 - The user scans the QR or opens the app and validates their PIN/biometrics.
 - The MyDigital ID app performs a 3-way handshaking process with the MyDOW Connector.
 - MyDOW verifies the user's X.509 digital certificate against an **OCSP (Online Certificate Status Protocol)** provider.
6. **Token Issuance:** MyDOW returns a JWT containing user claims (Nama and Nric) to MyID Access Server. MyID Access Server processes the claims, signs a standard OIDC Access/ID Token pair, and redirects back to the 3rd-party application.

2. Standard MyID Access Server Client Configurations

To configure a MyID Access Server Client for your application within the MyDigital ID Realm, apply the parameters below:

- **Client ID:** Unique identifier assigned to your application (e.g., my-app-client).
- **Client Protocol:** openid-connect.
- **Access Type:**
 - **Confidential:** For server-side apps capable of storing secrets safely (Laravel, Spring Boot, Express, Django).
 - **Public:** For SPAs or native mobile apps (React, Flutter, Android/iOS).
- **Valid Redirect URI:** <3rd Party App URL to handle authorization code>

Core Endpoints

Endpoint Name	Purpose	Endpoint URL Pattern
Authorization	Initiate OIDC authorization	https://<MYDID_HOST>/realms/<REALM>/protocol/openid-connect/auth
Token	Exchange Authorization Code for Access Token	https://<MYDID_HOST>/realms/<REALM>/protocol/openid-connect/token
UserInfo	Retrieve user identity claims	https://<MYDID_HOST>/realms/<REALM>/protocol/openid-connect/userinfo
Logout	Terminate SSO and local sessions	https://<MYDID_HOST>/realms/<REALM>/protocol/openid-connect/logout

3. Developer Implementation Guides

Implementation A: PHP 8 with Laravel Framework

1. Install Library:

```
composer require vizir/laravel-keycloak-web-guard:4.1.0
```

Publish vendor config: `php artisan vendor:publish`

`--provider="Vizir\KeycloakWebGuard\KeycloakWebGuardServiceProvider"`.

2. Environment .env:

```
KEYCLOAK_BASE_URL=https://sso.myid.my
KEYCLOAK_REALM=mydid
KEYCLOAK_CLIENT_ID=your-client-id
KEYCLOAK_CLIENT_SECRET=your-client-secret
KEYCLOAK_REDIRECT_URI=http://127.0.0.1:8000/mydid-callback
```

3. Configure Guard & Provider (config/auth.php):

```
'guards' => [
    'web' => [
        'driver' => 'keycloak-web',
        'provider' => 'keycloak_users',
    ],
],
'providers' => [
    'keycloak_users' => [
        'driver' => 'keycloak-users',
        'model' => App\Models\User::class,
    ],
],
```

4. Update User Model (User.php):

Add nama and nric to protected \$fillable:

```
protected $fillable = ['nama', 'nric', 'email', 'name'];
```

5. Define Routes (routes/web.php):

```
use Vizir\KeycloakWebGuard\Controllers\AuthController;
```

```
Route::group(['middleware' => 'keycloak-web'], function {
    Route::get('/dashboard', [DashboardController::class, 'index'])->name('dashboard');
    Route::get('/mydid-login', [AuthController::class, 'login'])->name('keycloak.login');
    Route::get('/mydid-callback', [AuthController::class, 'callback'])->name('keycloak.callback');
    Route::get('/mydid-logout', [AuthController::class, 'logout'])->name('keycloak.logout');
});
```

Implementation B: PHP 8 with CodeIgniter Framework

1. Install Library:

```
composer require jumbojett/openid-connect-php
```

2. Login Initialization (Auth.php Controller):

```
use Jumbojett\OpenIDConnectClient;

public function loginWithKeycloak
{
    $oidc = new OpenIDConnectClient(
        getenv('KEYCLOAK_BASE_URL') . '/realms/' . getenv('KEYCLOAK_REALM'),
        getenv('KEYCLOAK_CLIENT_ID'),
        getenv('KEYCLOAK_CLIENT_SECRET')
    );

    $oidc->setRedirectURL(getenv('KEYCLOAK_REDIRECT_URI'));
    $oidc->authenticate();

    $userInfo = $oidc->requestUserInfo();

    // Query local user using MyDigital ID NRIC claim
    $user = $this->auth_model->where('ic_number', $userInfo->nric)->first();

    if ($user) {
        session()->set('keycloak_user', ['nama' => $userInfo->nama, 'nric' => $userInfo->nric]);
        return redirect()->to('/Main');
    } else {
        session()->set('keycloak_user', ['nama' => $userInfo->nama, 'nric' => $userInfo->nric]);
        return view('auth/ic_number_not_found');
    }
}
```

3. Handle Global SSO Logout:

```

public function logout
{
    $idToken = session()->get('keycloak_id_token');
    session()->destroy();

    if ($idToken) {
        $logoutUrl = getenv('KEYCLOAK_BASE_URL') . "/realms/" . getenv('KEYCLOAK_REALM') .
        "/protocol/openid-connect/logout?" . http_build_query([
            'id_token_hint' => $idToken,
            'post_logout_redirect_uri' => base_url('/'),
            'client_id' => getenv('KEYCLOAK_CLIENT_ID'),
        ]);
        return redirect()->to($logoutUrl);
    }
    return redirect()->to('/login');
}

```

4. Network & Security Requirements

HTTPS / TLS Encryption

All client-to-server communications and redirect URIs must strictly use HTTPS/TLS for security.

Whitelisted Redirect URIs

A valid, secured Redirect URI (KEYCLOAK_REDIRECT_URI) hosted on your application to handle the incoming authorization code.

Outbound Network Connectivity

Your server environment must be able to reach the MyDigital ID SSO Host endpoints over standard web protocols (HTTPS / Port 443).

5. Database Schema Requirements

To store and link authenticated user identity data provided via JWT tokens, your user database model must include or support the following fields:

NRIC / MYKAD Number

To store or match the user's Malaysian IC number.

NAMA / MYKAD Name

To store the user's official full name returned by the identity provider.

6. Ecosystem Integration Best Practices

App Existence Check Module

Mobile webviews cannot natively verify if the MyDigital ID native application is installed on the device. Implement one of the following methods:

- **Option 1: App Presence Detection (Preferred):** Use custom URL scheme verification (e.g., mydigitalid://) via cross-platform bridge code. If the app is missing, block login initiation and display a modal prompting the user to install MyDigital ID first.
- **Option 2: Pre-Login Modal / Notice:** If the operating environment blocks deep-link schema checking, display an explicit pre-login notice informing users that an active, registered MyDigital ID installation is required prior to attempting login.

Login Compatibility

To ensure login compatibility across a wide range of mobile devices when integrating MyDigital ID Login into your mobile application, developers must implement the OpenID Connect (OIDC) flow using embedded webview rather than an in-app system browser or custom tab (such as Chrome Custom Tabs or SFSafariViewController).

Production Security Requirements

- **SSL/TLS:** All callback routes, API communications, and Keycloak domains must strictly operate over HTTPS.
- **Claim Hashing:** When storing NRICs in third-party databases, verify whether raw NRICs or cryptographic hashes (e.g., SHA-256) are required by your system's data governance standards.